

HOMWORK 1: POLICY GRADIENTS

CMU 10-703: DEEP REINFORCEMENT LEARNING (FALL 2026)

OUT: Monday August 31st, 2026

DUE: Friday September 11th, 2026 by 11:59pm EST

Instructions: START HERE

Note: This homework assignment requires a significant implementation effort. Please plan your time accordingly. General tips and suggestions are included in the “Guidelines on Implementation” section at the end of this handout.

- **Collaboration policy:** You may work in groups of up to four people for this assignment. It is also OK to get clarification (but not solutions) from books or online resources after you have thought about the problems on your own. You are expected to comply with the University Policy on Academic Integrity and Plagiarism¹.
- **Late Submission Policy:** You are allowed a total of 10 grace days for your homeworks. However, no more than 2 grace days may be applied to a single assignment. Any assignment submitted after 2 days will not receive any credit. Grace days do not need to be requested or mentioned in emails; we will automatically apply them to students who submit late. We will not give any further extensions so make sure you only use them when you are absolutely sure you need them. See the Assignments and Grading Policy here for more information about grace days and late submissions: https://cmudeeprl.github.io/703website_f26/
- **Submitting your work:**
 - **Gradescope:** Please write your answers and copy your plots into the provided LaTeX template and upload a PDF to the Gradescope assignment titled “Homework 1.” Additionally, zip all code folders into a directory titled `<andrew_id>.zip` and upload it to the Gradescope assignment titled “Homework 1: Code.” Each team should upload only one copy of each part. Regrade requests can be made within one week after the assignment is graded.

This is a challenging assignment. **Please start early!**

¹<https://www.cmu.edu/policies/>

Introduction

The goal of this homework is to give you experience implementing and analysing Deep Reinforcement Learning algorithms. We'll start with one of the oldest Reinforcement Learning algorithms, REINFORCE, then build our way up to N -step Advantage Actor-Critic (A2C). Although these algorithms were proposed many years ago, they serve as the foundation of many current state-of-the-art approaches.

Problem 0: Collaborators

Please list your name and Andrew ID, as well as those of your collaborators.

Problem 1: Policy Gradient Algorithms (48 pts)

Problem 1.1: REINFORCE (10 pts)

We begin this homework by implementing episodic REINFORCE [4], a policy-gradient RL algorithm. Use the conda environment defined in `code/environment.yaml` for compatibility. Please write your code in `code/pg/pg.py`; the template code should give you an idea of how to structure your code for this problem and the next two problems.

In [3], the authors present a version of REINFORCE that performs a single gradient update to the policy for every timestep of a collected episode. Here, we ask you to consider a slightly different version that instead performs a single policy gradient update for every *episode* of collected data, treating the entire episode as a minibatch of experience when determining the policy gradient. By performing a single gradient update per episode, we can obtain policy gradient estimates with (slightly) less variance, and we reduce the risk of significantly changing the policy if we encounter a particularly long episode of experience, which generally tends to result in learning instabilities. We also ask you to use the Adam optimiser instead of performing vanilla gradient ascent. If you're not familiar with Adam, you can think of it as a fancy version of gradient descent that introduces momentum and an adaptive learning rate for each dimension. Pseudocode for the required version of REINFORCE is provided in Algorithm 1. Recommended hyperparameter values are included in the code template. The network settings and hyperparameters for the policy are provided to you in `code/pg/pg.py`.

Evaluate your implementation of Algorithm 1 on the `CartPole-v1` environment by running 5 IID trials² with $E = 3,500$. During each trial, freeze the current policy every 100 training episodes and run 20 independent test episodes. Record the mean **undiscounted** return obtained over these 20 test episodes for each trial and store them in a matrix $D \in \mathbb{R}^{\text{number of trials} \times \text{number of frozen policies per trial}} = \mathbb{R}^{5 \times (3,500/100)} = \mathbb{R}^{5 \times 35}$. Note that each entry

²We ask you to run multiple trials as DRL algorithms tend to exhibit a great deal of random variation both across implementations and across random seeds for a given implementation. This is well documented in the literature [1] and we encourage you all to reflect on the various sources of randomness that make DRL algorithms hard to compare.

Algorithm 1 REINFORCE

```
1: procedure REINFORCE
2:   Start with policy network  $\pi_\theta$ 
3:   repeat for  $E$  training episodes:
4:     Generate an episode  $S_0, A_0, R_0, \dots, S_{T-1}, A_{T-1}, R_{T-1}$  following  $\pi_\theta(\cdot)$ 
5:     for  $t = 0, 1, \dots, T - 1$ :
6:        $G_t = \sum_{k=t}^{T-1} \gamma^{k-t} R_k$ 
7:        $L(\theta) = -\frac{1}{T} \sum_{t=0}^{T-1} G_t \ln \pi_\theta(A_t | S_t)$ 
8:       Update  $\pi_\theta$  using Adam ( $\nabla_\theta L(\theta)$ )
9:   end procedure
```

in D is an average of 20 values, and any particular column of D gives us a “snapshot” of your implementation at some point in time during training on **CartPole-v1**.

In one figure, plot the **mean of the mean undiscounted return** on the y -axis against **number of training episodes** on the x -axis. In other words, plot the average of the entries in columns of D on the y -axis. On the same figure, also plot a shaded region showing the maximum and minimum mean undiscounted return against number of training episodes, where the max and min are taken across trials (i.e., show the max and min of each column in D). The plotting format is given to you in `code/pg/pg.py`.

In RL, the term “expected return” refers to $\mathbb{E}_{\tau \sim \pi_\theta} [\sum_{t=0}^{\infty} \gamma^t R_t]$, where the expectation is taken with respect to the randomness in the MDP (including the initial state, state transitions, action selection, and reward generation) under policy π_θ . When we ask you to plot the mean of the mean undiscounted return, the first mean refers to the randomness in the DRL algorithm (e.g., random weight initialisation), and the second mean refers to the randomness in the underlying MDP itself. When we look at a particular column of D , the variability among these 5 values primarily reflects randomness in the DRL training procedure (e.g., random initialization and sampled training experience), although each value also contains finite-sample evaluation noise because it is estimated from only 20 test episodes. For the purposes of comparing DRL algorithms, this variability is of critical importance.

Runtime Estimation: To help you plan your time, note that our implementation of Algorithm 1 takes 8 minutes to complete a single trial with $E = 3,500$ on a laptop.

Problem 1.2: REINFORCE with Baseline (10 pts)

An important early development of REINFORCE was the introduction of a baseline. In this part of the homework, we ask you to make a small change to your implementation of REINFORCE to examine the impact of this on the same **CartPole-v1** test environment.

Again, we ask you to consider a slightly modified version of the algorithm presented in Sutton & Barto’s text. The pseudocode is provided in Algorithm 2. For the baseline, we recommend making changes to the output layer of the network provided in `code/pg/pg.py`. Generate the same plot as for REINFORCE (i.e., run 5 IID trials and plot the mean, max and min of the mean undiscounted returns across trials). Take $E = 3,500$.

Runtime Estimation: Note that our implementation of Algorithm 2 takes 10 minutes to complete a single trial with $E = 3,500$ running on a laptop.

Algorithm 2 REINFORCE with Baseline

```

1: procedure REINFORCE WITH BASELINE
2:   Start with policy network  $\pi_\theta$  and baseline network  $b_\omega$ 
3:   repeat for  $E$  training episodes:
4:     Generate an episode  $S_0, A_0, R_0, \dots, S_{T-1}, A_{T-1}, R_{T-1}$  following  $\pi_\theta(\cdot)$ 
5:     for  $t = 0, 1, \dots, T - 1$ :
6:        $G_t = \sum_{k=t}^{T-1} \gamma^{k-t} R_k$ 
7:        $L(\theta) = -\frac{1}{T} \sum_{t=0}^{T-1} (G_t - b_\omega(S_t)) \ln \pi_\theta(A_t | S_t)$ 
8:        $L(\omega) = \frac{1}{T} \sum_{t=0}^{T-1} (G_t - b_\omega(S_t))^2$ 
9:       Update  $\pi_\theta$  using Adam ( $\nabla_\theta L(\theta)$ )
10:      Update  $b_\omega$  using Adam ( $\nabla_\omega L(\omega)$ )
11: end procedure

```

Problem 1.3: N -step Advantage Actor Critic (20 pts)

Another important development of REINFORCE was to choose the baseline to be an approximate state-value function and use it to bootstrap estimates of the return. We call such a baseline a *critic*, and it carves out a family of algorithms depending on the degree of bootstrapping performed. See Chapter 13 of the Sutton & Barto text for more details. In this part of the homework, we ask you to implement this algorithm for a variety of different bootstrapping strategies. In particular, we ask you to implement the version of N -step A2C given in Algorithm 3.

Once again, please use the provided network architectures and hyperparameters.

Please provide 3 separate figures similar to those in the previous problems for $N = 1, 10, 100$. For each value of N , please run 5 IID trials as in the previous parts of the homework. Take $E = 3,500$ for each value of N .

Runtime Estimation: Note that our implementation of Algorithm 3 takes 11 minutes to complete a single trial with $E = 3,500$ and $N = 100$ on a 2020 MacBook Pro.

Algorithm 3 *N*-step Advantage Actor-Critic

```
1: procedure N-STEP ADVANTAGE ACTOR-CRITIC
2:   Start with actor network  $\pi_\theta$  and critic network  $V_\omega$ 
3:   repeat:
4:     Generate an episode  $S_0, A_0, R_0, \dots, S_{T-1}, A_{T-1}, R_{T-1}$  following  $\pi_\theta(\cdot)$ 
5:     for  $t = 0, 1, \dots, T - 1$ :
6:       
$$V_{\text{end}} = \begin{cases} V_\omega(S_{t+N}) & \text{if } t + N < T \\ 0 & \text{otherwise} \end{cases}$$

7:       
$$G_t = \left( \sum_{k=t}^{\min(t+N-1, T-1)} \gamma^{k-t} R_k \right) + \gamma^N V_{\text{end}}$$

8:       
$$L(\theta) = -\frac{1}{T} \sum_{t=0}^{T-1} (G_t - V_\omega(S_t)) \ln \pi_\theta(A_t | S_t)$$

9:       
$$L(\omega) = \frac{1}{T} \sum_{t=0}^{T-1} (G_t - V_\omega(S_t))^2$$

10:      Update  $\pi_\theta$  using Adam ( $\nabla_\theta L(\theta)$ )
11:      Update  $V_\omega$  using Adam ( $\nabla_\omega L(\omega)$ )
12: end procedure
```

Problem 1.4: *N*-step A2C & REINFORCE with Baseline (4 pts)

How does *N*-step A2C relate to REINFORCE and REINFORCE with Baseline? That is, under what conditions do these algorithms become equivalent?

Problem 1.5: REINFORCE with & without a Baseline (4 pts)

Does adding a baseline improve performance? Briefly explain why you think this happens.

Problem 2: Question Answering (12 pts)

1. (2 pts) Consider standard policy iteration in a finite tabular MDP with known dynamics. Assume that at every iteration, policy evaluation is performed exactly until convergence, followed by exact policy improvement.

In standard policy improvement, the new policy selects an action that maximizes $q_\pi(s, a)$. Suppose instead that the new policy selects an action that maximizes the advantage

$$A_\pi(s, a) = q_\pi(s, a) - v_\pi(s).$$

Which of the following statements is correct?

- Policy iteration using q_π converges to an optimal policy, but policy iteration using A_π need not.
- Policy iteration using A_π converges to an optimal policy, but policy iteration using q_π need not.
- Neither procedure is guaranteed to converge to an optimal policy.
- Both procedures converge to an optimal policy.

Select one and justify your answer.

2. (2 pts) Two different policies $\pi_1 \neq \pi_2$ can have exactly the same action-value function,

$$q_{\pi_1}(s, a) = q_{\pi_2}(s, a) \quad \text{for all } (s, a).$$

Answer true/false and justify in 1–3 sentences.

3. (2 pts) Suppose $Q_\phi(s, a)$ and $V_\psi(s)$ are represented by two neural networks and are learned from a finite set of trajectories. It is always the case that

$$V_\psi(s) = \sum_a \pi(a | s) Q_\phi(s, a)$$

for every state s . Answer true/false and justify in 1–3 sentences.

4. (2 pts) Suppose π^* is an optimal policy for a finite MDP, so that

$$v_{\pi^*}(s) = v^*(s) \quad \text{for every state } s.$$

Then its action-value function also satisfies

$$q_{\pi^*}(s, a) = q^*(s, a) \quad \text{for every state-action pair } (s, a).$$

Answer true/false and justify in 1–3 sentences.

5. (4 pts) Recall that the advantage function is defined as

$$A_\pi(s, a) = q_\pi(s, a) - v_\pi(s).$$

- (a) Prove that for every state s ,

$$\mathbb{E}_{a \sim \pi(\cdot|s)} [A_\pi(s, a)] = 0.$$

- (b) Explain why

$$q_\pi(s, a) > 0$$

does not necessarily imply

$$A_\pi(s, a) > 0.$$

- (c) Give an example or brief explanation of how an action with a large positive return, for example $q_\pi(s, a) = 100$, could nevertheless have negative advantage and therefore appropriately receive a negative weight in the policy-gradient estimator.

Additional Questions [Not Graded]

The questions in this section are not graded. However we highly encourage thinking deeply about the solutions and the associated concepts for better internalizing the material.

1. Consider the discounted finite-horizon objective

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r(s_t, a_t) \right].$$

Suppose we use the policy-gradient estimator

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \frac{1}{T} \sum_{t=0}^T \nabla_\theta \log \pi_\theta \left(a_t^{(i)} \mid s_t^{(i)} \right) G_t^{(i)},$$

where

$$G_t^{(i)} = \sum_{k=t}^T \gamma^{k-t} r \left(s_k^{(i)}, a_k^{(i)} \right)$$

is the discounted reward-to-go from $(s_t^{(i)}, a_t^{(i)})$.

Is $\hat{g}$ an unbiased estimator of $\nabla_\theta J(\theta)$? Justify your answer.

2. Give an example of a setting in which temporal-difference (TD) methods can perform worse than Monte Carlo methods for policy evaluation. Explain why TD learning may fail or introduce systematic error in your example, while Monte Carlo evaluation may still behave reasonably.

Hint: Consider what can happen when the state representation violates the Markov property.

3. Suppose we want to jointly learn the advantage function $A^\pi(s, a)$ and the state-value function $V^\pi(s)$ using TD learning and neural-network function approximation.
 - (a) Propose a neural-network parameterization that represents both $A^\pi(s, a)$ and $V^\pi(s)$.
 - (b) Describe a TD-style objective that could be used to train this network from transitions (s_t, a_t, r_t, s_{t+1}) .
 - (c) Your parameterization should ensure that the learned advantage and value functions are mutually consistent. Explain what constraint an advantage function must satisfy and how your parameterization can enforce this constraint.

Hint: Advantage functions satisfy a consistency constraint under the policy's action distribution.

4. In Problem 1.3, the N -step A2C actor uses $G_t - V_\omega(S_t)$ as an estimate of the advantage. Consider the one-step case, $N = 1$, and suppose the critic is exact:

$$V_\omega(s) = V^\pi(s).$$

Then the actor's advantage estimate is the one-step TD error

$$\delta_t = r_t + \gamma V^\pi(s_{t+1}) - V^\pi(s_t).$$

Prove that

$$\mathbb{E}[\delta_t \mid s_t = s, a_t = a] = A^\pi(s, a).$$

In words, explain what this result says about the one-step TD error as an estimator of the true advantage.

5. Suppose the learned critic has a constant error at every state:

$$\hat{V}(s) = V^\pi(s) + c,$$

where c is the same constant for all states. Consider the one-step TD advantage estimator

$$\hat{A}_t = r_t + \gamma \hat{V}(s_{t+1}) - \hat{V}(s_t).$$

- (a) Show that

$$\mathbb{E}[\hat{A}_t \mid s_t, a_t] = A^\pi(s_t, a_t) - (1 - \gamma)c.$$

- (b) The advantage estimator $\hat{A}_t$ is therefore biased. Does this necessarily imply that the resulting policy-gradient estimator is biased? Justify your answer.
- (c) Use the baseline identity

$$\mathbb{E}_{a \sim \pi(\cdot \mid s)} [\nabla_\theta \log \pi_\theta(a \mid s) b(s)] = 0$$

to resolve the apparent contradiction.

Feedback

Feedback: You can help the course staff improve the course by providing feedback. What was the most confusing part of this homework, and what would have made it less confusing?

Time Spent: How many hours did you spend working on this assignment? Your answer will not affect your grade.

Guidelines on Implementation

This homework requires a significant implementation effort. It is hard to read through the papers once and know immediately what you will need to implement. We suggest that you think about the different components (e.g., network definition, network updater, policy runner, ...) that you will need to implement for each of the methods, and then read through the papers with these components in mind. By this we mean that you should divide the implementation into small components with well-defined functionality rather than try to implement everything at once. Much of the code and experimental setup is shared between the different methods so identifying well-defined reusable components will save you trouble.

A couple of points which may make life easier:

- Debuggers such as `pdb` or built-in IDE debuggers are **extremely** useful as we start to consider more sophisticated implementations.
- Consider dumping your data (the matrix D) after every trial instead of generating the required plots in the same script.

Some hyperparameter and implementation tips and tricks:

- For efficiency, you should try to vectorize your code as much as possible and use **as few loops as possible**. For example, in lines 5 and 6 of Algorithm 1 (REINFORCE) you should not use two nested loops. How can you formulate a single loop to calculate the cumulative discounted rewards? Hint: Think backwards!
- We recommend using a discount factor of $\gamma = 0.99$.

References

- [1] Cédric Colas, Olivier Sigaud, and Pierre-Yves Oudeyer. How many random seeds? statistical power analysis in deep reinforcement learning experiments. *arXiv preprint arXiv:1806.08295*, 2018.
- [2] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.
- [3] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- [4] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. In *Machine Learning*, pages 229–256, 1992.